CLAWBADA · HERMES HANDOFF

Master Handoff to Hermes

2026-05-20 · From outgoing engineer-of-record → Codex 5.5

CONTENTS

> **Audience.** This document is the canonical onboarding artifact for **Hermes**, the Codex 5.5 agent being trained to operate as CEO + CTO of the Clawbada project. It is **not** a public GitBook (that audience is human players). This is a deep, structured pass-off that assumes a senior technical reader, agrees with the constraints in `.claude/CLAUDE.md` / `AGENTS.md` / `SOUL.md`, and wants to push the project to testnet then mainnet.
>
> **How to read this document.** - Parts 1–3 = the *what* and *why* (strategy, game design, tokenomics). Read first. - Parts 4–5 = the *how* (architecture, repo layout, conventions). Read in full before touching code. - Part 6 = section-by-section technical status. Each subsection has **Desired Outcome** (where it must end up to ship) and **Current State** (exactly what's there now). Use this as the punchlist. - Part 7 = active backlog and prioritized path to launch. Drives day-to-day work. - Part 8 = operating procedures (memory, sweeps, commits, deployments).
>
> **Truthfulness contract.** This document is accurate as of **2026-05-19**. Where the doc says "DONE", the corresponding code is in the working tree. Where it says "DEFERRED" or "OPEN", the corresponding work has not started or has partial coverage as noted. Status fields are mirrored from `~/.claude/projects/-Users-alepore-Clawbada/memory/launch-blockers.md`, which is the live tracker.

Part 0 — Quick orientation for Hermes

You are inheriting a single-developer codebase that has been built fast over ~3 months. The architecture decisions are intentional. Do **not** re-litigate them without strong cause. In priority order:

1. **The two CRITICAL P0 production blockers (X1, X2) are CLOSED.** The operator-worker series (PR-A foundation, PR-B create_battle, PR-C resolve_round + settle accounting) is the largest server-side lift and is shipped. See Part 6.3.
2. **The campaign style is "find every bug before users do".** Each PR ships with adversarial Codex sweeps. Findings are tracked in `launch-blockers.md` until DONE or explicitly DEFERRED with a written rationale. Continue this discipline.

3. **Project-wide biggest blocker is Unity / assets / playtesting**, not testnet deploy. The active production battle viewer is the Unity project at `packages/battle-engine/ClawbadaBattle/` (see Part 6.8). Designer arena/lobster/UI/VFX/audio assets need to land, then a WebGL build, then bridge verification, then real gameplay playtesting — before exposing testnet users to anything. **Sepolia deploy is the chain/backend blocker** (Part 7.2) and is what gates end-to-end *chain* validation, but it is not the project-wide top item.
4. **Operate cautiously.** Read `.claude/CLAUDE.md`, `AGENTS.md`, `SOUL.md`, and `COMMANDS.md` before doing anything destructive. Match the existing terse-output style. Never invent test coverage you didn't actually run.

The single most useful day-1 action is to read this document end-to-end (start with Part 6 if pressed for time), then `git log --oneline -50` and `git status` for ground-truth context.

Part 1 — Strategic overview

1.1 What Clawbada is

Clawbada is an **agent-first idle game on Base** (Coinbase's Ethereum L2). The design is inspired by Crabada (the abandoned 2022 Avalanche P2E) but corrects its terminal flaws (15:1 mint-to-burn, 12× annual ROI, trivially bottable, dependent on new-player inflow).

Players assemble teams of three **lobster NFTs** (ERC-1155) and compete in two parallel economies:

- **Idle Mining** — inflationary, low-risk, passive. Deploys a team for a 4-hour expedition; earns a fixed $CLAW reward.
- **Battle Mode** — zero-sum, active, hex-grid tactical PvP with ATB initiative-bar combat. Players wager $CLAW; winner takes the pot minus a 10% protocol fee.

At a ~63-65% battle win rate the two modes have roughly equal EV; above 65%, battle dominates. Mining emissions halve each 60-day season, so the game's center of gravity shifts toward battle as the seasons progress.

1.2 Who the players are

CLASS	DISCOVERY	WALLET	INTERFACE
Primary: OpenClaw AI agents	Moltbook (agent social network, ~1.5M registered agents)	Bankr.bot (Privy server wallets) / MoltX.io / any EOA / ERC-4337 smart wallet	Contract ABI + REST/WebSocket API
Secondary: Humans	Base App	SignInWithBase (EIP-4361) → ERC-4337 smart wallet with paymaster	React/Next.js web UI

"Agent-first" is operational, not aspirational. **The API and contract ABI are the primary product surface.** The web UI is a courtesy layer for humans on the Base App mini-app. Every architectural decision (rate limits, auth, idempotency keys) must remain agent-friendly first.

1.3 Why this can work

Three observations Crabada missed:

1. **The economy must be net deflationary at steady state.** Emissions schedule + dual-mode economy (battle is zero-sum redistribution; only emissions inflate) achieves this. Mint-to-burn target <1:1.
2. **Bot-resistance must be designed in, not bolted on.** Battle mode's tactical depth (10 classes × hex positioning × move-action selection) raises the cost of trivial scripting; sophisticated agents are *fine* — the goal is to reward sophisticated strategy.
3. **Agents are not adversaries; they are the target customer.** A protocol that publishes a clean API + skill package and gets distributed via Moltbook is built for them.

1.4 OpenClaw ecosystem integration

```
OpenClaw (agent OS — creation, memory, state management)
    ↓ deploys agent with budget via
Bankr.bot (wallet infra — Privy server wallets, instant provisioning)
    ↓ agent researches strategies on
MoltX / Moltbook (agent social network)
    ↓ agent pays fees via
x402 (Coinbase micropayment protocol — $0.0001 tx fees)
    ↓ agent plays Clawbada via
Base smart contracts + game API
```

Integration touchpoints (status):

- **OpenClaw skill package**: Publish a Clawbada skill to `BankrBot/openclaw-skills`. **Not started.**
- **Bankr.bot wallets**: Agents interact with @bankrbot on X to fund their game wallet. **No bespoke integration required; agents arrive with funded wallets.**
- **Moltbook presence**: Game events/results posted to Moltbook. **Not started.**
- **x402 micropayments**: Entry fees, breeding costs, tournament stakes. **Not started; on-chain $CLAW transfers used today, x402 wrapping is post-launch.**

1.5 Competitive positioning

Direct comparables are sparse. The closest analogues:

- **Crabada (Avalanche)** — dead. Lessons learned, no longer a competitor.
- **DeFi Kingdoms (Harmony/Klaytn)** — different genre (idle MMORPG); shows that on-chain idle works at scale but its tokenomics also broke.
- **Pixels (Ronin)** — farming sim; demonstrates Base/Ronin agent traction.
- **Roving generative agents on Moltbook** — not games per se; demonstrate the agent-as-customer thesis.

Clawbada's edge: **agent-first architecture + dual-mode economy + battle skill ceiling**. No competitor combines all three for Base.

Part 2 — Game design — complete mechanics

> Cross-reference: `.claude/CLAUDE.md` is the canonical game-design spec. `docs/gitbook/*.md` is the human-facing companion. This section is the technical-spec summary — when a number disagrees, **`.claude/CLAUDE.md` wins**.

2.1 Lobsters (the NFTs)

Each lobster is an ERC-1155 NFT carrying:

- **DNA** — packed `uint256`. Decoded below.
- **Class** — one of ten (Bulwark, Mantis, Leviathan, Tempest, Specter, Sentinel, Reaver, Abyss, Kraken, Ember).
- **Legend status** — 2 bits (normal / legend / 2 reserved values).
- **Evolution tier** — Base / Evolved / Elite / Apex.
- **Damage** — `uint8` 0-100, accumulated from battles; ≥80 blocks battle entry until repaired.
- **Purity score** — count of dominant alleles matching the lobster's overall class (0-6).

DNA encoding (256 bits, high to low)

```
[255:252]  Class                 4 bits   (0–9, 10 classes)
[251:250]  Legend                2 bits   (0=normal, 1=legend, 2–3 reserved)
[249:244]  Breed type            6 bits   (up to 64 visual subtypes)
[243:240]  Reserved              4 bits
[239:96]   6 body parts × 3 alleles × 8 bits  (144 bits)
           Each allele: class affinity (4b) + variant (4b)
[95:0]     Reserved              96 bits  (future mechanics)
```

Body parts and primary stat affinity:

SLOT	PART	PRIMARY STAT	VISUAL
0	Carapace	HP	Back shell, color, pattern
1	Claws	Attack	Claw shape, ornamentation
2	Tail	Speed	Tail fan length
3	Antennae	Critical	Length, shape, glow
4	Eyes	Armor	Eye stalks
5	Legs	HP	Leg style

Each part contributes to **all five stats** (HP / Attack / Armor / Speed / Critical); the affinity determines the strongest contribution.

2.2 The ten classes

#	CLASS	ROLE	MOVE RANGE (HEXES)	SPECIAL	IDENTITY (BASE HP/ATK/ARMOR/SPD/CRIT)
1	**Bulwark**	Tank	1	Fortify	700 / 70 / 120 / 80 / 90 — team damage -40% for 2 turns
2	**Mantis**	Assassin	3	Ambush	375 / 100 / 70 / 130 / 125 — ignore 50% armor
3	**Leviathan**	Bruiser	1	Crush	600 / 130 / 100 / 70 / 80 — highest single-target burst
4	**Tempest**	Nuker	3	Maelstrom	450 / 110 / 80 / 105 / 115 — AoE 3-hex
5	**Specter**	Debuffer	3	Haunt	425 / 85 / 85 / 125 / 120 — atk/armor -20% for 4 turns
6	**Sentinel**	Support	2	Rally	650 / 70 / 110 / 90 / 100 — heal ally 30% max HP
7	**Reaver**	DPS	2	Rend	475 / 120 / 80 / 110 / 95 — 40 bleed/turn × 6
8	**Abyss**	Lifesteal	2	Devour	525 / 110 / 90 / 95 / 100 — damage = heal
9	**Kraken**	Controller	2	Bind	550 / 90 / 100 / 105 / 95 — stun 1 turn + 2-turn immunity
10	**Ember**	Glass Cannon	3	Inferno	350 / 140 / 60 / 100 / 130 — 4-hex range, caster takes 25% recoil

The **tournament graph is balanced**: each class beats 4 and loses to 4 (1.25× damage on advantage, 0.80× on disadvantage, 1.0× neutral). The exact 10×10 advantage matrix is in `packages/game-logic/src/classes.ts`.

2.3 Stats and combat math

Stats scale: **+20% / +40% / +60% per evolution tier**, **+10% for legend**, plus body-part modifiers from DNA. HP is scaled ×5 in battle for 24–36-turn pacing.

Attack damage:

```
damage = 100 × min(Attack/Armor, 2.2) × class_mult × crit_mult × distance_mult × VRF

  100             = Attack base power
  Attack/Armor    = ratio, capped at 2.2× (prevents one-shots)
  class_mult      = 1.25 (advantage) | 1.0 (neutral) | 0.80 (disadvantage)
  crit_mult       = 1.5 (crit) | 1.0
  crit chance     = Critical / (Critical + 200)
  distance_mult   = 1.0 (adjacent) | 0.75 (2 hex) | 0.50 (3 hex) | miss (4+)
  VRF             = drand variance, uniform [0.85, 1.15]
```

Defend: halves incoming damage until lobster's next turn; counters adjacent attackers for 30-base; no counter vs. Specials. Yields +2 charge.

Move: reposition within class movement range; no damage; +1 charge (+2 if Move-then-Defend).

Special:

```
damage = special_base × min(Atk/Armor, 2.2) × class_mult × purity_mult × VRF
purity_mult = 1 + 0.10 × purity_score   (×1.0 at 0 match → ×1.6 at 6/6)
enhanced_chance = 5% + 5% × purity_score   (5% → 35%)
Costs 3 charge (consumes all).
Range per class table above. Defend halves Special damage; doesn't counter.
```

ATB initiative bar: all 6 lobsters share a single tick tracker. Next-turn tick = `prev_tick + (1000 / effective_speed)`. Speed is clamped to `[0.5×, 1.5×] × base`. Stun immunity for 2 turns after a stun expires.

Win condition: team wipeout. Hard cap: 100 total turns with HP% tiebreak (rarely reached).

Randomness: single drand beacon rolled at TEAM_REVEAL seeds the entire battle's RNG stream. Same beacon → reproducible battle (essential for the S2 on-chain replay path).

2.4 Teams

- 3 lobsters per team. Unlimited team slots per wallet.

- Lobsters are **locked** when committed to a team, mining, or in an active battle. Cannot list on marketplace or transfer.
- Duplicate classes allowed (mono-class is valid but suboptimal due to shared weaknesses).
- Team Power = sum of evolution-tier weights (Base=0 / Evolved=1 / Elite=2 / Apex=3). Range 0–9.

2.5 Two-mode economy

2.5.1 Mining (idle)

MINE TIER	REQUIREMENT	WEIGHT	REWARD AT BASEREWARD=1250
Base	All 3 lobsters ≥ Base	1×	1,250 $CLAW
Evolved	All 3 lobsters ≥ Evolved	3×	3,750 $CLAW
Elite	All 3 lobsters ≥ Elite	10×	12,500 $CLAW
Apex	All 3 lobsters ≥ Apex	25×	31,250 $CLAW

- **Fixed per-expedition rewards**: `baseReward × tierWeight`, locked at expedition start. No pro-rata.
- **Season budget cap**: enforced via `SeasonBudgetExhausted` revert when a new expedition would exceed `totalEmission`.
- **Admin-tunable** **`baseReward`** mid-season via `setBaseReward()` (SEASON_ADMIN_ROLE).
- 4-hour expeditions across all tiers (6/day per team).
- No diminishing returns per wallet (flat reward regardless of fleet size).
- $CLAW stake required for expeditions, except the first one for faucet lobsters.

2.5.2 Battle Mode (active)

BRACKET	STAKE	COMBINED POT	PROTOCOL FEE	WINNER NET	LOSER NET
Low	2,500	5,000	500	+2,000	-2,500
Mid	10,000	20,000	2,000	+8,000	-10,000
High	50,000	100,000	10,000	+40,000	-50,000

- Entry gate: **all 3 lobsters Evolved-tier or higher**.
- Protocol fee: 10% of combined pot → Treasury.sol → 85% burn / 15% dev.
- Battles use **ATB initiative-bar combat** (LOKR-style) with full information during play. Only team composition is hidden via on-chain commit-reveal.
- ~3–5 minute typical match. 24–36 total turns.
- Breakeven win rate (including repair): ~58%. Mining-equivalent at ~63–65%.

2.5.3 Power Matchmaking (V3 S1)

- Match by **Team Power bucket** (3–9 integer sum, Evolved=1/Elite=2/Apex=3) × **stake bracket** → up to 21 sub-pools.
- **Adaptive radius expansion** to prevent thin-pool starvation:
  - 0–30 s: exact power match
  - 30–60 s: ±1 power
  - 60–120 s: ±2 power
  - 120 s+: any power within stake bracket (HUD warns of mismatch)
- Random pairing within sub-pool at launch; ELO matchmaking deferred to S1.5.
- The **2-minute Deposit window is the consent mechanism** — the player can walk away if the opponent's power doesn't suit them.

2.5.4 Battle lifecycle (8 phases — contract enum)

```
None=0 → Deposit=1 → TeamCommit=2 → TeamReveal=3 → Active=4 → AwaitingFinalize=5
                                                                      ↓
                                                                Settled=6
                                                                     OR
                                                                Cancelled=7
```

1. **Matchmaking** (off-chain) — API ticker pairs players.
2. **Stake Deposit** (on-chain) — both players call `BattleArena.deposit(battleId)` with stake + 5% anti-grief.
3. **Team Commit-Reveal** (on-chain) — commit hash, then reveal teamId + salt. Hash binds `(battleId, msg.sender, teamId, salt)`.
4. **VRF Beacon** (on-chain) — one drand beacon rolled at TEAM_REVEAL; submitted via `BattleVRF.submitBeacon` for replay reproducibility.
5. **Battle** (off-chain via WebSocket, server-authoritative) — ATB bar runs. Each lobster's turn = optional Move + one Action (Attack/Defend/Special). 60s shot clock; auto-Defend on timeout.
6. **Settlement** (on-chain) — operator submits `(battleId, winner, damages)` to `BattleArena.settle()`. Phase becomes `AwaitingFinalize` and `BattleProposed` event fires.
7. **Dispute window** (on-chain, optional) — loser may `disputeBattle(battleId, evidence)` with a 10% bond. Windows per bracket: 5 min / 30 min / 1 h. Rate-limited to 5/24h per address.
   - **S1**: `adminResolveDispute()` — DEFAULT_ADMIN_ROLE multisig, 24h SLA.
   - **S2** (roadmap): `BattleResolver.replay()` — on-chain deterministic re-execution from `{initial state + VRF beacon + ordered turn submissions}`.
8. **Repair** (on-chain) — both players call `RepairShop.repair(lobsterId, points)`. $CLAW burned. Lobsters ≥80 damage blocked from battle until repaired.

2.5.5 Repair system

OUTCOME	DAMAGE POINTS
Winner	5–15 (VRF)
Loser	20–40 (VRF)

TIER	COST ($CLAW PER DAMAGE POINT)
Evolved	5
Elite	15
Apex	40

- Instant repair (no cooldown), partial repairs allowed.

- All repair costs are $CLAW burns through Treasury.sol (85/15 split).
- Damaged lobsters can still mine — damage only gates battle entry.

2.6 Breeding

- 2 parents → 1 offspring (always Base tier, always tradeable).
- 5 breeds per parent lifetime max. 48-hour cooldown per parent after each breed.
- Offspring generation = `max(parentA.gen, parentB.gen) + 1`.
- Parents NOT consumed (unlike evolution fuel).
- All breeding fees routed through Treasury.sol.

Per-parent cost:

```
per_parent_cost = 500 × breed_multiplier × 1.5 ^ parent_generation
breed_multiplier by parent's breed count: [1, 1.5, 2.5, 4, 8]   // 1st through 5th breed
```

Gene inheritance (per body part):

1. **Primary selection** — each parent contributes 1 allele: Dominant 50% / R1 33% / R2 17%.
2. **Third allele** — VRF picks one parent; one of that parent's two remaining alleles is drawn equiprobably. No mutations in S1.
3. **Ordering** — sort the 3 alleles by (class-match priority, variant value). Highest → Dominant, next → R1, lowest → R2.

Result: ~3–4 generations of selective breeding to reach 5–6 purity.

Class inheritance: 50/50 from either parent (VRF). Same-class parents = guaranteed class.

2.7 Evolution

TIER UP	FUEL	$CLAW COST	UNLOCKS	STAT BOOST
Base → Evolved	2 Base	2,000	Evolved mine + Battle Mode	+20%
Evolved → Elite	2 Evolved	10,000	Elite mine	+40%
Elite → Apex	2 Elite	50,000	Apex mine	+60%

- Fuel lobsters are **burned** (NFT sink).
- $CLAW cost is burned through Treasury.sol.
- Cost to reach Apex: 26 Base-tier lobsters burned + ~62K $CLAW per Apex.

2.8 Legends

- **~0.3% chance per breed** (VRF roll at offspring creation).
- **+10% base stats** stacked with evolution + legend visuals.
- **Not hereditary** — each breed is an independent roll.
- Faucet lobsters cannot be legends (only bred offspring can).
- No purity bonus, no exclusive access — pure prestige + modest stat edge.

2.9 Cold-start faucets

Two faucets, both **closing 6 days 23 hours after launch**:

- **Lobster Faucet** — 5 random lowest-class lobsters, soulbound.
- **$CLAW Faucet** — 7,000 $CLAW (requires holding 5 soulbound lobsters).

Wallet eligibility:

- ≥ 0.001 ETH balance.
- Wallet age ≥ 7 days on Base.
- ≥ 3 prior transactions on Base before the 7-day mark.
- 1 claim per wallet per faucet.

Sybil defense relies on the chained dependency (lobster → $CLAW), wallet age + tx history, soulbound preventing consolidation, and the hard ~7-day window.

Part 3 — Tokenomics — $CLAW

3.1 Supply and distribution

- **Fixed max supply: 1,000,000,000 $CLAW** (1B).
- **100% fair launch — no team/VC allocation.** Dev funded through the 15% protocol-fee share.

ALLOCATION	%	AMOUNT	PURPOSE
Mining emissions	70.5%	705M	Core distribution (gameplay earned)
DEX liquidity	12.5%	125M	Self-deployed Uniswap V3 ($CLAW/ETH, 0.3% fee tier)
Treasury	10.0%	100M	Protocol reserves, bug bounties, future game modes
Faucet pre-mint	7.0%	70M	~10K wallets × 7K drip

No airdrop. Self-deployed LP — no Clanker (1% fee considered too extractive for a high-frequency game token).

3.2 Emission schedule

```
Season 1  (days   1–60):   352.5 M $CLAW   ← gold rush
Season 2  (days  61–120):  176.25 M
Season 3  (days 121–180):   88.125 M
Season 4  (days 181–240):   44.06 M
Season 5  (days 241–300):   22.03 M
Season 6  (days 301–360):   11.02 M
Season 7+ (day  361+):       7.05 M / season    ← floor (perpetual)
```

- ~98.4% of mining pool emitted in year 1.
- Gold rush (S1–S2): 75% of total mining pool in first 4 months.

- Steady state from S7 onward: 7.05M per 60-day season.
- Each season: emission halving, leaderboard reset, class rebalancing (dev-controlled S1; data-driven from days 40–50 onward).

3.3 DEX liquidity — self-deployed Uniswap V3

- Pair: $CLAW/ETH on Uniswap V3 (Base), 0.3% fee tier.
- Seed: **125M $CLAW + 6 ETH** → ~$0.0001/$CLAW, ~$100K FDV at $2,100/ETH.
- Wide V3 range: ~5× downside (~$20K FDV) to ~5× upside (~$500K FDV).
- Operational ETH reserve: **3.5 ETH retained** (gas, emergency LP adjustments, deployments).
- Total ETH budget: **9.5 ETH** (~$20K at $2,100/ETH).
- LP fees stay in the ecosystem (no third-party extraction).

3.4 Economic model

- **Mining emissions** = sole inflationary source (fixed schedule, halving).
- **Battle Mode** = zero-sum redistribution; only the 10% protocol fee is burned.
- **All other protocol fees** (breeding, marketplace, repair, evolution) route through Treasury.sol.

Protocol fee split (everywhere):

RECIPIENT	SHARE	PURPOSE
Burn	**85%**	Deflationary pressure
Dev wallet	**15%**	Hosting, RPC, ongoing dev

- **No ve-CLAW.** No staking yield, no governance token. Earn only by playing (mining / battling / breeding-and-selling).
- Target mint-to-burn ratio: < 1:1 (net deflationary at steady state).

3.5 Token sinks

- Battle stakes (zero-sum redistribution + protocol-fee burn).

- Battle repair (every match burns $CLAW).
- Evolution costs (2K / 10K / 50K $CLAW per tier, all burned).
- Breeding fees (scale exponentially by generation).
- Lobster decay / feeding ($CLAW burn).
- Tiered mining access (indirect — requires evolution).
- Strategy tax (rapid successive actions cost escalating fees).

3.6 Locking mechanisms

- Mining stakes: locked during expedition.
- Battle stakes: locked during match + 5% anti-grief deposit.
- Lobster locking: any committed/active state prevents sell/transfer.

Part 4 — Technical architecture

4.1 Tech stack at a glance

LAYER	TECH	NOTES
Chain	Base (Ethereum L2, OP Stack, Chain ID 8453, 200 ms Flashblocks)	Inherent MEV resistance — no public mempool
Smart contracts	Solidity (ERC-20 $CLAW, ERC-1155 lobsters, game economy)	Foundry build
Agent interface	Contract ABI + REST/WebSocket API	Primary product surface
Human interface	React/Next.js 15 + wagmi + viem (Base App mini-app)	Secondary
Battle sim (logic)	TypeScript in `packages/game-logic` (deterministic; on-chain & off-chain parity)	Pure math, no rendering
Battle viewer (render)	Unity 6 WebGL project at `packages/battle-engine/ClawbadaBattle/`	Actively in design; `apps/web/src/lib/battle-anim/` is the parallel design rig
Auth (agents)	Bankr.bot wallet / ERC-4337 / any EOA	EIP-191 sig + replay window
Auth (humans)	SignInWithBase (Base Account SDK, EIP-4361 SIWE)	One click
Payments	x402 (Coinbase micropayment) — *post-launch wrapping*	$CLAW used directly today
RPC	Alchemy or QuickNode (Base endpoints)	Configurable per env
DB	Postgres 16	drizzle-orm, drizzle-kit migrations

LAYER	TECH	NOTES
Queue / locks	Postgres advisory locks + `FOR UPDATE SKIP LOCKED`	No Redis/Kafka
Rate limit	Redis-backed, per-wallet	3 tiers (general/faucet/combat)
Distribution	OpenClaw skill package, Moltbook, Base App	Pending

4.2 Architecture pattern — hybrid on-chain / off-chain

On-chain (trustless, permanent): $CLAW token, NFT ownership, breeding, staking, marketplace, treasury, team assignments, battle stakes, battle settlement, evolution, repair, VRF beacon store.

Off-chain (fast, cheap, iterable): combat resolution, mining timers, matchmaking, leaderboards, round resolution.

Players own their lobsters and tokens on-chain. Compute-heavy game logic runs off-chain with periodic on-chain settlement (every settle / advanceRound is a single tx).

4.3 Repo layout

```
Clawbada/
├── apps/
│   ├── api/         Hono API server — agent + human REST/WS surface, matchmaker tick
│   ├── engine/      Operator-worker (long-lived process: outbox poller, season
monitor, drand, battle resolver)
│   ├── indexer/     Chain event watcher (live + backfill, advances DB phase)
│   └── web/         Next.js 15 frontend (Base App mini-app)
├── contracts/       Solidity — Foundry build, 12 contracts (BattleArena, ClawToken,
…)
├── packages/
│   ├── asset-gen/   Procedural lobster art (genome → pixels), template editor v5
│   ├── battle-engine/  Unity 6 project (ClawbadaBattle/) — production battle viewer,
actively in design
│   ├── chain/       ABIs (extracted from Foundry), viem wallet/public clients,
contract helpers
│   ├── db/          drizzle schemas, migrations, schema barrel
│   ├── game-logic/  Pure TS: battle-sim, breeding, dna, evolution, classes, hash,
elo, types
│   └── logger/      pino wrapper
├── docs/
│   ├── HERMES_HANDOFF.md   ← this document
│   └── gitbook/     Human-facing docs (also covered in main README)
└── .claude/         Project conventions for AI agents (CLAUDE.md, AGENTS.md, SOUL.md,
COMMANDS.md, LEARNED.md)
```

4.4 Apps and what each one does

apps/api

Hono server. Exposes REST + WebSocket. Primary agent surface.

```
api/
├── game/
│   ├── mining/      start, status, claim
│   ├── combat/      queue (join/leave/status, pool-depth), battle-reads, battle-
writes (deposit, commit-team, reveal-team, commit-moves, reveal-moves, handle-timeout)
│   ├── breeding/    preview, breed request, offspring status
│   ├── teams/       create, assign, list, disband
│   ├── market/      list, buy, price history
│   ├── evolution/   request, status
│   ├── repair/      pay, history
│   ├── render/      lobster image render (procedural)
│   └── activity/    on-chain event feed
├── agent/           register, strategy hooks, events WS feed
├── faucet/          lobster faucet + $CLAW faucet
└── settlement/      batched on-chain settlement (legacy; superseded by engine
operator-worker)
```

Auth via `walletAuth` middleware (EIP-191 sig, asymmetric replay window: 5 min past, 30 s future).

Matchmaker ticker (`apps/api/src/lib/matchmaker/tick.ts`) runs inside the API process. Single-transaction match decisions with `FOR UPDATE SKIP LOCKED` on `matchmaking_queue` and a `pg_advisory_xact_lock(BATTLE_PREDICTION_LOCK_KEY)` around the simulate→insert sequence.

apps/engine

Long-lived operator-worker process. Five responsibilities today:

1. **Operator-worker** — durable outbox (`operator_jobs` table) drained at 1 s poll cadence. Handlers:
   - `create_battle` — submits on-chain `BattleArena.createBattle` via MATCHMAKER_PRIVATE_KEY wallet, verifies receipt, flips `battles.status = 1 (created)` .
   - `resolve_round` — replays state from `on_chain_events` MoveRevealed, calls game-logic resolver, writes `battle_rounds` , submits `advanceRound` or `settle` via RESOLVER_PRIVATE_KEY wallet.
2. **Season monitor** — 5-min poll loop; auto-rolls seasons via `MiningPool.startSeason()` when emission/duration triggers fire.
3. **Mining timer** — expedition completion notifications.

4. **drand beacon submitter** — submits VRF beacons to `BattleVRF` contract for replay reproducibility.
5. **Legacy BattleStateMachine** — kept as dead-but-imported (no live caller). Will be removed post-launch.

Operator-worker design (`apps/engine/src/operator/`):

- `worker.ts` — `OperatorWorker` class with `start()` , `stop()` (drain-with-timeout), `registerHandler()` . Polls `operator_jobs` with `FOR UPDATE SKIP LOCKED` at 1s. Backoff schedule `[5s, 30s, 5min, 1h]` , MAX_ATTEMPTS = 5.
- `types.ts` — `JobStatus` , `JobResult` discriminated union, `JobContext` with `recordTxHash(hash)` helper.
- `errors.ts` — `classifyError(err)` walks viem error cause chain for `ContractFunctionRevertedError` ; 25 known BattleArena custom-error names → `dead` (no retry); everything else → `transient` . `wrapHandler(fn)` higher-order helper.
- `jobs/create-battle.ts` — handler. Uses `getMatchmakerClient` .
- `jobs/resolve-round.ts` — handler. Uses `getResolverClient` . Replays from `on_chain_events` ; battle_rounds INSERT with `ON CONFLICT (battleId, round) DO NOTHING` .

apps/indexer

Chain event watcher. Live (`watchContractEvent`) + backfill (`getContractEvents` , batches of 2000 blocks). One watcher per contract subclassing `EventWatcher` .

`BattleWatcher` event list (`apps/indexer/src/watchers/battle-watcher.ts`): `BattleCreated` , `StakeDeposited` , `TeamCommitted` , `TeamRevealed` , `MoveCommitted` , `MoveRevealed` , **`BattleProposed`** (X12, phase=5), `BattleSettled` , `BattleCancelled` , `DamageApplied` , `AntiGriefSlashed` .

Phase advances are **non-regressing** via `lt(battles.phase, N)` UPDATE guards (prevents reorg-redelivery from overwriting Settled with an earlier state).

`BattleSettled` handler runs settle accounting in a `db.transaction` : wei→display conversion for winnerPayout/protocolFee, `settledAt IS NULL` idempotency guard, MAX(round) for totalRounds, agent upsert + ELO/wins/losses/totalBattles update.

`MoveRevealed` handler enqueues `resolve_round` operator job when both reveals are present.

apps/web

Next.js 15 frontend. Base App mini-app shape. wagmi + viem.

- `(game)/battle/[id]` — spectator battle page.
- `(game)/game/battle` — active player battle page (uses `BattleMoves` component).
- `(game)/game/breeding|evolution|mining|repair|teams|page` — game routes.
- `(game)/leaderboard`, `(game)/activity`, `(game)/lobster/[id]`, `(game)/market` — supporting pages.
- `app/faucet/page.tsx` — cold-start faucet (deleted in working tree per status; planning revival).
- `app/page.tsx` — landing (deleted in working tree; pending redesign — see "silly-roaming-catmull" plan).
- `app/agents/page.tsx` — agent docs (deleted; pending redesign).

Key hooks: `use-queue-state` (queue lifecycle reducer + WS + polling), `use-battle-ws` (reconnect + auth renewal), `use-auth` (EIP-191 sig caching, inFlight map).

4.5 Database schema (high level)

Tables (`packages/db/src/schema/`):

- `lobsters` — NFT-mirrored cache (DNA, owner, evolutionTier, damage, breedCount, generation, soulbound, locked, purity).
- `teams` — team composition.
- `expeditions` — mining expeditions.
- `battles` — battle metadata (incl. `queuedTeamA/B`, `powerA/B`, `status` for operator-worker lifecycle, `phase` for contract enum, `winner`, `protocolFee`, `winnerPayout`, `totalRounds`, `settledAt`).
  - CHECK constraint: `status IS NULL OR status IN (0,1,2,3,4)`.
- `battle_rounds` — per-round resolved state (`actions` JSONB with bigint→string serialization, `teamAHp/teamBHp`, `vrfSeed`).
  - UNIQUE INDEX on `(battle_id, round)`.
- `listings`, `price_history` — marketplace.
- `breeds` — breeding history.
- `seasons` — emission tracking.

- `agents` — agent profile + ELO + wins/losses/totalBattles.
- `matchmaking_queue` — Power Matchmaking queue (with `powerScore`, unique on `address`).
- `matchmaking_decisions` — telemetry (append-only).
- `indexer_state` — last block per watcher.
- `on_chain_events` — raw event log (no unique constraint yet; tracked as A5).
- `operator_jobs` — outbox for the operator-worker (`jobType`, `payload`, `idempotencyKey` UNIQUE, `status`, `attempts`, `lastError`, `nextAttemptAt`, `txHash`, `completedAt`).
  - CHECK constraint: `status IN (0,1,2,3)`.

Migrations under `packages/db/drizzle/`: `0000_steep_shadowcat.sql` (base), `0001_past_zaladane.sql` (idempotent V3 S1 + A2 queued_team_a/b), `0002_handy_marten_broadcloak.sql` (operator_jobs + battles.status), `0003_wide_sharon_carter.sql` (CHECK constraints), `0004_remarkable_talkback.sql` (battle_rounds UNIQUE).

4.6 Smart contracts

CONTRACT	LOC	ROLE
`ClawToken.sol`	ERC-20 $CLAW with emission schedule, halving, burn	
`LobsterNFT.sol`	ERC-1155 lobsters; DNA storage, metadata, batch transfers	
`TeamManager.sol`	Team assignment (3 per slot), lobster locking, unlimited slots	
`BreedingLab.sol`	Breed two lobsters → new lobster, DNA combination, fee burn	
`MiningPool.sol`	Stake team to mine, claim rewards, season management	
`Marketplace.sol`	Lobster trading, listing, fee collection (only unlocked lobsters)	
`Treasury.sol`	Protocol fee splitter — 85% burn / 15% dev wallet	
`Faucet.sol`	Temporary lobster faucet + $CLAW faucet (closeable by admin)	
`BattleArena.sol`	Battle lifecycle: stake deposit, team commit-reveal, settlement, dispute resolution, anti-grief	
`BattleResolver.sol`	Pure combat math library (placeholder; S1 doesn't replay on-chain)	
`BattleVRF.sol`	drand beacon store, beacon → randomness derivation	

CONTRACT	LOC	ROLE
`EvolutionLab.sol`	Lobster evolution: burn 2 fuel + $CLAW → 1 evolved lobster	
`RepairShop.sol`	Post-battle damage repair ($CLAW burn)	

Critical contract conventions:

- All custom errors named (no string reverts). Listed in `packages/chain/src/abis/battle-arena.ts`.
- `MATCHMAKER_ROLE` gates `createBattle`. `RESOLVER_ROLE` gates `advanceRound` + `settle`. `DEFAULT_ADMIN_ROLE` (multisig) gates `adminResolveDispute`. Mainnet `Configure.s.sol` grants these to **different addresses** (DeployHelpers.s.sol enforces `MATCHMAKER_ADDRESS != RESOLVER_ADDRESS`).
- `handleTimeout(battleId)` is permissionless and routes to the phase-specific cleanup.
- `safeERC20` migration applied across 8 contracts (audit I-03/I-04, March 2026).
- Self-purchase guard on Marketplace (I-01); redundant SSTORE removed in battle-arena (I-02).

4.7 Conventions Hermes must adopt

- **Bun is the workspace package manager.** `npm install` will not work. `~/.bun/bin/bun` needs to be in PATH for shell invocations.
- **Tailwind v4 + Next.js 15** in apps/web requires `@tailwindcss/postcss` + `postcss.config.mjs`. Put hex values directly in `@theme inline` (NOT `var()` indirection).
- **drizzle-kit generate** is the migration source of truth. Hand-written migrations (`0001_v3s1_*` and `0002_v3s1_*`) were merged into `0001_past_zaladane.sql` and deleted. Don't reintroduce hand-written migrations; declare constraints in the TS schema and let drizzle generate.
- **ABI extraction**: `packages/chain/scripts/extract-abis.ts` reads from `out/<Contract>.sol/<Contract>.json` (Foundry). Run after `forge build`. The ABI drift caused the A11 HIGH finding — Hermes should add the A12 CI gate.
- **Tests use** `bun:test` with `mock.module(...)` for module-level mocks. Real-DB integration tests are tracked as X9.

- **Code style**: terse, no over-engineering. Default to no comments unless the WHY is non-obvious. Don't reference current task/fix in code comments — that belongs in PR descriptions.
- **Memory system**: `~/.claude/projects/-Users-alepore-Clawbada/memory/` is the persistent memory. `MEMORY.md` is the index (one-line entries). Topic files are stored alongside. See Part 8.2 for the operating procedure.

Part 5 — What "DONE" actually means — campaign artifacts

The Clawbada codebase has been hardened via an aggressive multi-PR audit campaign over April–May 2026. Each PR carried adversarial Codex sweeps after every fix bundle until the floor was reached. The campaign's findings are tracked in `~/.claude/projects/-Users-alepore-Clawbada/memory/launch-blockers.md`.

Hermes should:

- **Read `launch-blockers.md` daily.** It is the live state of every open + closed audit finding.
- **Continue the sweep cadence**: for every fix bundle of meaningful scope, run a Codex adversarial sweep (`Agent → codex:codex-rescue`) with the write-to-file pattern. The sweep prompt template lives in the conversation history; reuse it.
- **Don't re-flag tracked items** in new sweeps unless the underlying assumption changed. Tracked items have explicit rationale.

Campaign sweep reports for the major PRs are stored at `/tmp/clawbada-*-codex-findings.md` and are read-only artifacts of the campaign. They are useful for understanding *why* a fix was applied a certain way.

5.1 PR series summary

SERIES	CLOSED	CROSS-CUTTING FINDINGS CLOSED
PR-A (operator-worker foundation)	Outbox table + worker scaffold + classify/wrap helpers	(foundation only)
PR-B (create_battle)	X1 (CRITICAL P0 — no createBattle handoff)	X3 (queued-team revalidation, folded), X8 (error classifier)
PR-C (resolve_round + settle accounting)	X2 (CRITICAL P0 — no resolver), X12 (BattleSettled accounting), X13 (handleTimeout UX)	HIGH-1 (role split), MEDIUM-1 (agent upsert)

All operator-worker work is now landed. 68 engine tests pass; all 5 packages typecheck clean (one pre-existing `dna.test.ts` overload error in game-logic is unrelated).

Part 6 — Section-by-section status

Each subsection follows the **Desired Outcome / Current State / Open Items** template. Status flags are: **DONE** | **PARTIAL** | **NOT DONE** | **DEFERRED**.

6.1 Smart contracts

Desired Outcome. All 12 contracts deployed to Base Sepolia for testnet; battle-tested via real agent + human flows for ≥ 2 weeks; deployed to Base mainnet with admin multisig + role separation (MATCHMAKER vs RESOLVER vs ADMIN); ABI auto-extracted and CI-gated; security-contact NatSpec on every contract.

Current State.

- **DONE**: All 12 contracts implemented. 464 Foundry tests pass (14 suites + 60 boundary tests). Marketplace zero-fee bug fixed. SafeERC20 migration across 8 contracts. C-01 security-contact NatSpec added. C-05/C-06 admin-roles runbook + contracts-audit CI workflow shipped. H-01 challenge window + V3 dispute bonding live.
- **NOT DONE**: Deployment to Base Sepolia (`Deploy.s.sol` + `Configure.s.sol` are ready but not run). No mainnet plan documented yet.

Open Items. - Launch-blockers **#2 + #3** — deploy + populate addresses. Unblocks everything downstream. - Tracker **A12** — CI gate on ABI freshness (prevents A11 recurrence). - Tracker **X15** — startup assertion that engine's derived signer addresses match on-chain role holders.

6.2 Off-chain game logic — `packages/game-logic`

Desired Outcome. All deterministic math (battle sim, DNA, breeding, evolution, repair, classes, ELO) lives in this package as pure TS. Importable by both apps and indexer without app boundaries. 100% unit-test coverage on math.

Current State.

- **DONE**: Battle resolver + battle-sim (initBattle, resolveRound, getWinner, isFinished, simulateBattle). DNA pack/unpack. Breeding cost + gene inheritance. Evolution rules. Repair cost formula. Class advantage matrix. Hash helpers. ELO (`calculateNewElo`).
- **PARTIAL**: Tests — 10 test files exist (battle-resolver, battle-sim, breeding, dna, evolution, gene-inheritance, repair, classes, hash, constants). One pre-existing overload error in `dna.test.ts` (unrelated to recent campaign work).

Open Items. - Cross-package consumers: indexer now imports `calculateNewElo` ; engine's `apps/engine/src/matchmaking/elo.ts` is now a re-export shim. Single source of truth.

6.3 Operator-worker — `apps/engine/src/operator`

Desired Outcome. Every operator-signed on-chain tx (createBattle, advanceRound, settle, season rollover) goes through a durable outbox + worker pattern with: idempotency keys, retry-with-backoff, dead status, crash-recovery via priorTxHash reconciliation, role-specific signer wallets (MATCHMAKER for create, RESOLVER for resolve/settle, OPERATOR for season/drand), error classification (contract revert → dead, RPC error → transient).

Current State (DONE — closes X1 + X2 + X3 + X8 + HIGH-1):

- `worker.ts` — `OperatorWorker` class. `FOR UPDATE SKIP LOCKED` claim. 1s polling. Stop-with-drain via `tickInFlight` + `stopping` flag (HIGH-A2). Restart-safe via stopping=false reset in start() (FU2). Crash recovery: `recoverStaleRunning()` on start.
- `types.ts` — `JobStatus`, `JobResult` discriminated union (`{ok:true,txHash?} | {ok:false,retry:'dead'|'transient',error}`), `JobContext.recordTxHash(hash)` with bounded internal retry [200ms, 1s, 5s] + `TxHashPersistError` sentinel.
- `errors.ts` — `classifyError(err)` walks viem cause chain, 25 BattleArena custom errors → dead. `wrapHandler(fn)` validates result shape (FU2 LOW-5 full union check), routes TxHashPersistError separately.
- `jobs/create-battle.ts` — handler. priorTxHash uses bounded `waitForTransactionReceipt` (never blind resubmits). Mints WS-equivalent signal via DB `battles.status=1` (X10 cross-process WS deferred). markCreateFailed re-throws DB errors for transient retry.
- `jobs/resolve-round.ts` — stateless replay from `on_chain_events.MoveRevealed`. battle_rounds INSERT with `onConflictDoNothing({target:[battleId,round]})`. State.finished → settle; else advanceRound. NO accounting writes (moved to indexer's BattleSettled).
- `index.ts` — registers `wrapHandler(createBattleHandler)` for `create_battle`, `wrapHandler(resolveRoundHandler)` for `resolve_round`.

68 tests pass across worker, errors, create-battle, resolve-round, seasons.

Open Items. - **X7** (DEFERRED, S2) — `recoverStaleRunning` is single-host only. Multi-instance needs `claimed_by` worker-token + heartbeat lease. - **X11** (DEFERRED) — Ops alert on `operator_jobs.status=dead` + reconciliation script for stuck `battles.status=0` rows. Pre-launch must address minimally. - **X14** (DEFERRED, S2) — BattleSettled idempotency claim is non-atomic under concurrent multi-instance indexer. Tie-in with X7. - **X15** (PARTIAL) — `.env.example` documents `MATCHMAKER_PRIVATE_KEY` + `RESOLVER_PRIVATE_KEY`; README + docker-compose still mention only `OPERATOR_PRIVATE_KEY`. Startup assertion against on-chain role holders pending.

6.4 Indexer — `apps/indexer`

Desired Outcome. Live + backfill chain event watcher per contract. Advances `battles.phase` to mirror contract enum. Triggers operator-worker jobs (resolve_round, ...). Performs canonical accounting at finality (BattleSettled → battles + agents). Reorg-aware with confirmation depth.

Current State.

- **DONE**: `BattleWatcher` with non-regressing phase advances (StakeDeposited → 2, TeamCommitted → 3, TeamRevealed → 4 + teamA/B writes, **BattleProposed → 5**, BattleSettled → 6, BattleCancelled → 7). MoveRevealed → enqueues resolve_round outbox job (idempotency_key='resolve_round:battleId:round'). BattleSettled handler does canonical accounting (winnerPayout/protocolFee wei→display, totalRounds from MAX(round), agent upsert + ELO via `calculateNewElo` in a `db.transaction`).
- **DONE (A7, A9, A10)**: lazy module-scope arena cache for `readBattleForPhase`; warn-log on chain-read failure; non-regressing UPDATE `WHERE lt(phase, N)` guards.
- **NOT DONE**: Reorg handling (X5).

Open Items. - **X5** (DEFERRED) — No block-hash tracking, no confirmation depth, no removed-log handling. Reorg drops `BattleCreated` / `Settled` / `Cancelled` → DB stays. Fix needs confirmation depth OR block-hash store + rollback/replay. - **X14** (DEFERRED, S2) — non-atomic settle claim under multi-instance. - **A5** (DEFERRED) — `on_chain_events` lacks unique index on `(tx_hash, log_index)`. Retry inserts duplicates. Frontend activity feed reads duplicates. - **A8** (DEFERRED, NIT) — `WatcherConfig.events` allowlist is dead code (viem isn't fed it). Rename to `archivedEvents` or feed to viem.

6.5 API — `apps/api`

Desired Outcome. Hono-based server exposing the full agent surface (REST + WS). Battle lifecycle endpoints, matchmaker tick, calldata builders for every contract write, agent registration, faucet, market, breeding, evolution, mining, repair, leaderboard, activity feed, render. Auth via EIP-191 sig (5-min past, 30-s future replay window). Rate-limited per-wallet (3 tiers).

Current State.

- **DONE**: 160 tests across 16 files. All routes covered (teams, faucet, mining, breeding, market, evolution, repair, agent, leaderboard, activity, combat, render). Hono `walletAuth` middleware. F-2F asymmetric replay window. F-2A `WS_AUTH_LIFETIME_SEC`. Combat queue + battle-reads + battle-writes routes. **`POST /api/game/combat/:battleId/handle-timeout`** calldata endpoint (X13). `GET /api/game/combat/:battleId/my-team` for caller-private queued team ID. `redactPrivateBattleFields` on public reads. Matchmaker outbox-insert in same tx as battles+queue (PR-B). `/queue/status` returns `recentBattle` (latest non-failed) + `failedRecentBattle` (status=4) per the FU2 fix.

- **DONE**: `readBattle` returns `phaseDeadline` + `payoutDeadline` + `disputed` for X13 handleTimeout button visibility.
- **PARTIAL**: A2c — battle row with NULL queued_team_a/b returns from `/my-team` and currently renders as "repair-needed" UI. Ops runbook for backfill not written.

Open Items. - **A2b** (DEFERRED, MEDIUM) — No tests for `/my-team` endpoint (participant A/B/non-participant/unknown), `/history` and `/:battleId` redaction, matchmaker queuedTeamA/B writes. - **A4** subsumed into **X13** (DONE).

6.6 Frontend — `apps/web`

Desired Outcome. Base App mini-app that delivers the full game loop for humans: queue → match → deposit → commit → reveal → battle → settle → repair → re-queue. Auto-recoverable from WS drops, wallet switches, browser restarts (commitment salt persistence is a noted open). Tight integration with operator-worker status (pending_create / create_failed UX). Real-time battle animation (canvas, 35 files / ~5900 LOC, post-launch upgradable to sprite sheets).

Current State.

- **DONE (A1, A2)**: Commit hashes correctly bind `msg.sender` for both team and move commits. Frontend uses `/my-team` for the pre-reveal teamId source. SessionStorage keys scoped by `(battleId, round, address)` (no cross-wallet leak). Wallet-switch reset effect dispatches `reset` + clears `rehydratedRef`. Queued polling at 3s with stable deps `[state.kind, auth?.isConnected, address]`. `failedRecentBattle` handling.
- **DONE (X13)**: `HandleTimeoutAction` banner shows when `isTimeoutable(chain)` returns true. Suppresses CTA on disputed AwaitingFinalize (LOW-01) and final-round both-reveals at MAX_ROUNDS (LOW-02).
- **DONE (PR-B X1)**: Pending-create UI on battle page when `db.status=0`; create-failed UI when `db.status=4`. Battle page renders correct branch BEFORE the generic chain-null pending branch.
- **DONE (PR 6, F-X3 etc)**: Battle move animations — canvas-based viewer, 6 animations, 10 class VFX, choreography state machine, particle physics. Integrated into spectator + active battle pages.
- **NOT DONE**: Landing page (`app/page.tsx`), agents page (`app/agents/page.tsx`), faucet page (`app/faucet/page.tsx`) all deleted from working tree pending the redesign plan at `~/.claude/plans/silly-roaming-catmull.md`.

Open Items. - **A3** (DEFERRED, MEDIUM) — sessionStorage commitment fragility. Salts + moveData stored only per-tab. Browser restart → reveal falls back to `''` → revert. Fix: wallet-signature-derived deterministic salt OR durable per-wallet/per-battle storage. Sizable redesign. - **A6** (DEFERRED, LOW) — Wagmi sender pinning during commit/reveal. Account switch mid-flow → poisoned commit. Add pre-submission assertion. - **X4** (DEFERRED, MEDIUM) — Battle action UI ignores `chain.phase`. `BattleMoves` derives phase from booleans; AwaitingFinalize/Cancelled/Settled-without-winner can still render commit/reveal → tx reverts. Fix: top-level switch on `chain.phase` first. - Frontend redesign (silly-roaming-catmull.md plan) — 6 phases, approved, not yet implemented.

6.7 Asset generation — `packages/asset-gen`

Desired Outcome. Procedural lobster art pipeline: genome → composited PNG. Designer-built body-part templates per class. Template editor for production designers. Integration into the API's `/api/game/render` endpoint.

Current State.

- **DONE**: Compositor pipeline. Color pipeline (mutations, palette shifts, breed-type-shifts, class-palettes — 11-role palette system after the March 2026 overhaul). Template editor v5 (`packages/asset-gen/tools/template-editor-v5.html`) with mutation zone preview, scale selection, universal outline role 10, undo-with-floating-selection.
- **PARTIAL**: Lobster image templates — 0 of 60 created (6 body parts × 10 classes). Procedural generation works without them but visual quality is placeholder.

Open Items. - **#11 + #17** — Need 60 template PNGs from designer. Pre-launch visual polish.

6.8 Battle engine package — `packages/battle-engine`

Desired Outcome. Unity 6 WebGL build that renders the live battle on the `/battle/[id]` page. React drives game state in via `SendMessage` ; Unity calls back via `JSBridge.jslib` on player commits. WebGL artifacts ship to `apps/web/public/unity-build/` and are loaded by `react-unity-webgl` .

Current State.

- **ACTIVELY IN DESIGN** — the user is working inside the Unity Editor (Unity 6 / 6000.4.2f1, Web Build Support). The project lives at `packages/battle-engine/ClawbadaBattle/`. See its `README.md` for the open/build flow.
- **Scripts compile-ready** (all C#, in `Assets/Scripts/`):
  - `Bridge/BattleBridge.cs` — React ↔ Unity messaging, JSON data classes.
  - `Bridge/JSBridge.jslib` (in `Plugins/WebGL/`) — JS interop for WebGL builds.
  - `Grid/HexGrid.cs`, `HexTile.cs`, `HexCoord.cs` — 6×5 pointy-top offset hex system, offset↔cube math, range queries, LKR-style on-demand tile spawning with 4 highlight states (stone = in range, blue = selected, red = enemy target, green = ally target).
  - `Battle/BattleManager.cs` — phase state machine (positioning/combat), round management, per-phase 60 s timer.
  - `Editor/ArenaAuthoringTool*.cs` — in-editor arena layout authoring + JSON exporter.
- **React side wired up** — `apps/web/src/lib/battle-anim/` is the *animation rig / prototype* used to design choreography, easing, palette FX, particles per Special. Lives alongside (not in place of) the Unity viewer. The battle page is configured to load `unity-build/` artifacts via `react-unity-webgl`.
- **Asset directories scaffolded** (empty, awaiting designer drops): `Art/Arenas/{Evolved,Elite,Apex}/`, `Art/Characters/`, `Art/HexTiles/`, `Art/Obstacles/`, `Art/UI/`, `Prefabs/Lobsters/`, `Prefabs/VFX/`, `Audio/Music/`, `Audio/SFX/`.
- **WebGL build target**: Brotli compression, 256 MB memory, output to `../../apps/web/public/unity-build/` (`battle.loader.js`, `battle.data.br`, `battle.framework.js.br`, `battle.wasm.br`).

Open Items. - Drop arena art into `Art/Arenas/{Evolved,Elite,Apex}/` and lobster sprites into `Art/Characters/`. - First end-to-end WebGL build + load test on `/battle/[id]` — confirm `BattleBridge` SendMessage round-trip works in browser (not just Unity Editor). - Wire the WebGL output path into the web app's build pipeline (currently a manual Unity export step). - Decide whether the animation rig in `apps/web/src/lib/battle-anim/` stays as a design tool post-launch or gets pruned once Unity ships.

6.9 Asset pipeline + on-chain image render

Desired Outcome. `GET /api/game/render/:lobsterId` returns the composited PNG, suitable for ERC-1155 `tokenURI` metadata. Caching strategy is per-DNA (DNA changes only on breeding/evolution → cache aggressively).

Current State.

- **DONE**: API endpoint exists. Composes from genome via packages/asset-gen.
- **NOT DONE**: ERC-1155 `tokenURI` metadata integration. No on-chain ipfs hash; off-chain rendering only.

Open Items. - Decide: stay off-chain (cheap, fast iteration) vs. lock metadata onto IPFS / Arweave for permanence (post-launch).

6.10 Faucet (cold start)

Desired Outcome. Two-step onboarding: claim 5 random soulbound lowest-class lobsters → claim 7,000 $CLAW drip. Anti-Sybil via wallet age + tx history + ETH balance + chained dependency. Hard cutoff ~7 days post-launch.

Current State.

- **DONE (contract side)**: `Faucet.sol` deployed in build, supports both flows + admin close.
- **NOT DONE**: Frontend faucet page (`apps/web/src/app/faucet/page.tsx`) was deleted pending redesign.
- **NOT DONE**: Sybil-defense checks (wallet age, tx history, ETH balance) — verify these are in `Faucet.sol` or `api/faucet/*` ; not confirmed.

Open Items. - Restore faucet page in the post-redesign frontend. - Confirm Sybil defenses match the design spec.

6.11 Marketplace, breeding, evolution, repair, mining

Desired Outcome. Each has a contract, API route, and frontend page. Lobster-locking enforced in TeamManager.

Current State.

- **DONE (contract side)**: All five contracts deployed in build. Marketplace I-01 self-purchase guard. Treasury fee routing applied to all five.
- **DONE (API side)**: All five route files exist with calldata builders + GET endpoints.
- **DONE (frontend pages)**: All five game pages exist (`/game/breeding` , `/game/evolution` , `/game/mining` , `/game/repair` , `/game/teams` , `/game/page` for dashboard).

Open Items. None at the contract/server layer that are unique to these modules; surface them in the frontend redesign.

6.12 Tokenomics + emission infra

Desired Outcome. $CLAW deployed; LP seeded ($CLAW/ETH, V3 0.3% fee tier); season schedule active; emission halving + floor enforced; Treasury fee routing burn/dev split working.

Current State.

- **DONE (contract side)**: ClawToken.sol implements emission schedule, halving, burn. MiningPool.sol enforces season budget cap (`SeasonBudgetExhausted` revert). Treasury.sol enforces 85/15 split.
- **DONE (engine side)**: Season monitor auto-rolls via `MiningPool.startSeason()` in 5-min poll loop. 12 tests.
- **NOT DONE**: Contracts not deployed anywhere. No LP seeded.

Open Items. - #2 + #3 — deploy to Base Sepolia. - Mainnet LP seed (post-testnet validation) — 6 ETH + 125M $CLAW with 3.5 ETH reserve.

6.13 OpenClaw skill package + Moltbook integration

Desired Outcome. Clawbada skill module published to `BankrBot/openclaw-skills`. Moltbook game events / results posts on settle.

Current State.

- **NOT DONE**: Neither integration started.

Open Items. - Post-mainnet priority. Will require coordination with OpenClaw + Moltbook teams.

Part 7 — Path to launch — prioritized backlog

> Live tracker: `~/.claude/projects/-Users-alepore-Clawbada/memory/launch-blockers.md`.

7.1 Project-wide biggest blocker — Unity / assets / playtesting

> Locked 2026-05-20: this is the project-wide top blocker, ahead of testnet deploy. Saved as durable memory.

1. **Designer pass — arena / lobster / UI / VFX / audio assets.** Drop into the scaffolded directories under `packages/battle-engine/ClawbadaBattle/Assets/`: `Art/Arenas/{Evolved,Elite,Apex}/`, `Art/Characters/`, `Art/HexTiles/`, `Art/Obstacles/`, `Art/UI/`, `Prefabs/Lobsters/`, `Prefabs/VFX/`, `Audio/Music/`, `Audio/SFX/`. Currently empty; this is the gate.
2. **First WebGL build.** Unity → File → Build Settings → Web → Brotli compression, 256 MB memory, output to `../../apps/web/public/unity-build/`. Expected artifacts: `battle.loader.js`, `battle.data.br`, `battle.framework.js.br`, `battle.wasm.br`.
3. **Bridge verification on `/battle/[id]`.** Confirm `react-unity-webgl` loads the build and the React ↔ Unity round-trip works end-to-end: React `SendMessage` → `BattleBridge.cs` → `BattleManager` → callbacks via `JSBridge.jslib` → `window.__clawbada.*`. Editor-mode parity is not sufficient — must run in browser.
4. **Gameplay playtesting.** Validate battle readability, pacing, target selection, movement, specials, timers, UX. Tune before exposing testnet users. The pure-math engine in `packages/game-logic` is solid; the open question is whether the *experience* lands.

7.2 Chain / backend blocker (gates end-to-end chain validation)

5. **#2 + #3 — Deploy contracts to Base Sepolia** + populate addresses across `.env`, app envs, and `packages/chain/src/addresses.ts`. Run `Deploy.s.sol` then `Configure.s.sol`. Verify role separation (MATCHMAKER ≠ RESOLVER on testnet via deployer fallback per `DeployHelpers.s.sol`).
6. **#13 — Vercel deployment config** for `apps/web`. Needs `vercel.json` (or `vercel.ts`) + env vars. `apps/web/package.json` build script needs verification.
7. **A12 — CI gate on ABI freshness**. Add CI step that runs `forge build` + `bun run extract-abis` and fails if `git diff --exit-code packages/chain/src/abis` is non-empty. Prevents A11 from recurring silently.

7.3 Pre-testnet (high value but not blocking)

8. **A2b — Test coverage gaps for A2**. Unit/integration tests for `/my-team`, redaction, matchmaker queuedTeamA/B writes.
9. **A2c — Ops runbook for NULL queued teams**. Pre-launch ops doc + reconciliation script.
10. **X11 — Alert on operator_jobs.status=dead**. Minimal: pino log → external alert. Pre-launch must address.
11. **X4 — Frontend phase gating**. `BattleMoves` should switch on `chain.phase` first.

7.4 Pre-mainnet

12. **#11 + #17 — Lobster art templates** (60 PNGs for the procedural NFT renderer; separate workstream from Unity arena art). Designer-blocked. Visual polish.
13. **A5 — `on_chain_events` unique index** migration. Stops duplicate event rows from leaking to the activity feed.
14. **X10 — Cross-process WS bridge** engine→API. Postgres LISTEN/NOTIFY pattern (~80–120 LOC). Reduces latency from ~3s polling to sub-second.
15. **X15 — Mainnet env vars + startup assertion**. README + docker-compose + engine startup check that derived signer addresses match on-chain role holders.

16. **#22 — Operator key failover** (SPOF). Currently single MATCHMAKER + single RESOLVER key. Plan a backup signer flow.

7.5 Pre-mainnet (S1 hardening — deferred but pre-launch)

17. **X5 — Reorg handling in indexer**. Confirmation depth or block-hash + reorg-detect.
18. **A3 — sessionStorage commitment durability**. Wallet-signature-derived deterministic salt OR durable per-wallet/per-battle storage.
19. **#19 — Pre-commit hooks** (husky + lint-staged).
20. **#20 — Monitoring/metrics**. Prometheus or equivalent. /health is the only signal today.
21. **A6 — Wagmi sender pinning**. Pre-submission assertion.

7.6 Post-launch / S2

- **X7 — Multi-instance worker lease/heartbeat**.
- **X14 — Atomic settle claim**.
- **A2d — Corrupt queue row validation + DB check constraint**.
- **A2e — Migration drift `DO $$` postcondition checks**.
- **A8 — `WatcherConfig.events` allowlist cleanup**.
- **#21 — Season rebalancing tools** (class win-rate analyzer, etc.).
- OpenClaw skill package publish.
- Moltbook event integration.
- Battle replay (BattleResolver.replay() on-chain) — removes admin from dispute resolution.

7.7 Deferred / S2-S3

- Procedural battle arena generation (S2-S3 enhancement).
- ELO-weighted matchmaking (S1.5; random pairing within bucket at launch).
- Cancel-rate throttling.
- ERC-1155 metadata permanence on IPFS/Arweave.

Part 8 — Operating procedures

8.1 Memory system — how Hermes should use it

The persistent memory at `~/.claude/projects/-Users-alepore-Clawbada/memory/` is the project's long-term brain. Hermes inherits it. Conventions:

- **`MEMORY.md` is an index** of one-line pointers. ≤ 200 lines. Anything longer goes in a topic file.
- **Topic files** are stored alongside, named `project_*.md`, `feedback_*.md`, `reference_*.md`, or by section (e.g. `launch-blockers.md`).
- **Memory types** (see Hermes's own equivalent of CLAUDE.md memory rules):
  - `user` — about the user's role, preferences, knowledge.
  - `feedback` — guidance from the user about how to approach work. Lead with the rule, then `**Why:**` and `**How to apply:**`.
  - `project` — ongoing initiatives, bugs, decisions. Same `Why/How to apply` structure.
  - `reference` — pointers to external systems.
- **Never store**: code patterns, file paths, git history, debugging recipes already in commits. Memory is for *what the code can't tell you*.
- **Refresh against reality** before quoting: a memory that names a function or file is a claim about *when it was written*. Verify before recommending action.

Key topic files in scope today:

FILE	USE
`launch-blockers.md`	The single source of truth for what's open/closed/deferred. Update on every fix.
`project_battle_v2_redesign.md`	The V3 S1 battle redesign spec.
`project_adversarial_audit_campaign.md`	Sweep cadence + standing tracker.
`project_character_animation_system.md`	Canvas-based battle animation architecture.
`battle-animation-engine.md`	Animation rig V2 spec (palette FX, choreography state machine).
`battle-server-wiring.md`	API/engine/indexer wiring map.
`reference_api_deployment.md`	External system pointers.
Various `feedback_*.md` files	User-provided guidance. Read once, internalize.

8.2 The audit-sweep workflow (continue this)

For any fix bundle of meaningful scope:

1. Apply the fix(es).
2. Run typecheck + tests (`bun run typecheck --filter '@clawbada/...'` per affected package; `bun test` in `apps/engine` for engine tests).
3. Launch a **Codex adversarial sweep**:
   - Use the `Agent` tool with `subagent_type: 'codex:codex-rescue'`.
   - Prompt template: "You are the standing adversarial reviewer for the Clawbada audit campaign. Write final report to `/tmp/clawbada-<topic>-codex-findings.md` via `cat <<'EOF' …EOF`. Artifact must survive."
   - Include: what changed, tracked deferrals (DO NOT RE-FLAG), adversarial probes, verdict format.
4. Read the report. Fix findings inline or track in `launch-blockers.md`.
5. Re-sweep until floor reached (verdict = "No new findings" or equivalent).
6. Update `launch-blockers.md`.

The Codex companion runs as a long-lived `codex app-server` process. If you suspect it's stuck, check `ps aux | grep codex-companion`. A typical sweep takes 5–25 minutes wall time depending on scope. The output file path is the truth signal — if it exists and has a verdict, the sweep finished. The companion task ID returned by the Agent tool is the handle for follow-up `SendMessage` calls.

8.3 Commits

Per `CLAUDE.md`: **NEVER commit unless explicitly asked**. When asked:

- Don't amend; create new commits.
- Don't use `--no-verify`.
- Don't `git add .` or `git add -A` — stage by file.
- Commit message style: `feat(...)` / `fix(...)` / `chore(...)` etc., 1–2 line summary, then explanation.
- Trailing line: `Co-Authored-By: Claude Opus 4.7 (1M context) <noreply@anthropic.com>` (or the equivalent for Hermes's identity).
- Always pass commit messages via HEREDOC for formatting safety.

8.4 Deployment

Today's deployment story is **incomplete**. The pieces:

- Contract deploy: `forge script` against Base Sepolia RPC. Scripts at `contracts/script/Deploy.s.sol` + `Configure.s.sol` + `DeployHelpers.s.sol`. Mainnet requires `MATCHMAKER_ADDRESS != RESOLVER_ADDRESS`.
- API + engine + indexer: 4 Dockerfiles in `docker/`. `docker-compose.yml` orchestrates Postgres 16-alpine + Redis + migrations service + the 4 apps.
- Web: needs Vercel config (#13).
- Env vars: `.env.example` is the template. Mainnet requires `MATCHMAKER_PRIVATE_KEY` + `RESOLVER_PRIVATE_KEY` in addition to `OPERATOR_PRIVATE_KEY`.

When Hermes deploys to testnet, the milestone gate is: a real agent (or human) can complete the full loop — register → queue → match → deposit → commit → reveal → battle → settle → repair → re-queue — without intervention.

8.5 Code style and patterns Hermes must match

- **Bun-first**. `~/.bun/bin/bun` PATH-prefixed for shell.
- **drizzle-orm** patterns: `db.transaction(async (tx) => ...)`, `eq()`, `and()`, `or()`, `inArray()`, `sql<T>\ ...` `. FOR UPDATE SKIP LOCKED via .for('update', { skipLocked: true })`.
- **viem** patterns: `getPublicClient(testnet)`, `getMatchmakerClient/getResolverClient/getOperatorClient(testnet)`. ABIs cast `as any` is acceptable.
- **Hono** patterns: middleware composition, `catchErrors` wrapper, `walletAuth` middleware, `c.req.json<T>()`.
- **React** patterns: TanStack Query with explicit `queryKey` including wallet address for participant-scoped data. useReducer for state machines (`use-queue-state.ts` is the reference). useRef for stable cross-render values + ref-based guards. Stable primitive deps in useEffect.
- **Error classification**: viem errors → walk cause chain → match against `PERMANENT_REVERT_NAMES`. Use `wrapHandler` for operator-worker handlers.

8.6 Standing decisions Hermes should not re-litigate

These were debated and committed. New evidence required to revisit:

- **Hybrid on-chain/off-chain**, not full on-chain.
- **drand for VRF** (not chain-native randomness).
- **Postgres + drizzle**, not Mongo / Supabase / Hasura.
- **Hono** for the API, not Express / Fastify / NestJS.
- **Next.js 15** for the frontend, not Remix / SvelteKit.
- **Foundry** for contracts, not Hardhat.
- **viem + wagmi**, not ethers.
- **Lobsters** (not crabs, not other animals).
- **10 classes** (not 8, not 12).
- **6 body parts × 3 alleles** DNA encoding.
- **60-day seasons** with halving, 7.05M floor.
- **70.5% mining / 12.5% LP / 10% treasury / 7% faucet** allocation.

- **85% burn / 15% dev** fee split.
- **3-lobster teams**.
- **ATB initiative bar** (LOKR-style), not turn-based or simultaneous.
- **Power Matchmaking** (V3 S1) — random within bucket at launch; ELO deferred to S1.5.
- **Single beacon per battle** for VRF.
- **MATCHMAKER vs RESOLVER role separation** on mainnet.

8.7 What to read on day 1

In order:

1. This document.
2. `.claude/CLAUDE.md` (project conventions for AI agents).
3. `.claude/AGENTS.md`, `.claude/SOUL.md`, `.claude/COMMANDS.md`, `.claude/LEARNED.md`.
4. `~/.claude/projects/-Users-alepore-Clawbada/memory/launch-blockers.md` (live tracker).
5. `~/.claude/projects/-Users-alepore-Clawbada/memory/MEMORY.md` (index → topic files).
6. `contracts/BattleArena.sol` (the most complex contract; understand the H-01 challenge window + V3 dispute bonding).
7. `apps/engine/src/operator/{worker,errors,types,jobs/create-battle,jobs/resolve-round}.ts` (the operator-worker series).
8. `apps/indexer/src/watchers/battle-watcher.ts` (full battle lifecycle indexer).
9. `apps/api/src/lib/matchmaker/match.ts` (single-tx matchmaker with outbox insert).
10. `apps/web/src/hooks/use-queue-state.ts` (queue lifecycle state machine).

8.8 First three weeks suggestion

Week 1. Deploy to Base Sepolia (#2 + #3). End-to-end test one full battle loop with a single deployer key (testnet fallback). Verify operator-worker drains correctly. Verify indexer phase advances mirror chain. Verify frontend handles the pending_create → created flip.

Week 2. A12 CI gate, A2b tests, A5 events dedupe migration, X11 ops alert. Address X4 (frontend phase gating) — this will trip during real testnet usage when battles go through AwaitingFinalize.

Week 3. A3 (sessionStorage durability) — wallet-signature-derived salts. X10 (cross-process WS bridge). Lobster art templates if the designer has shipped them. Begin the frontend redesign (silly-roaming-catmull plan).

Part 9 — Reference appendix

9.1 Glossary

- **AwaitingFinalize** — contract phase 5; the H-01 dispute window. `settle()` proposes; `finalizeBattle()` or `adminResolveDispute()` resolves; `BattleSettled` event fires on resolution.
- **BPS** — basis points. `BATTLE_PROTOCOL_FEE_BPS = 1000` = 10%. Denominator is 10,000.
- **Backoff schedule** — `[5s, 30s, 5min, 1h]` per failed operator-worker attempt; after 5 attempts → dead.
- **drand** — distributed randomness beacon. Used as VRF seed for battles.
- **EIP-191** — personal_sign signature format used for API auth.
- **EIP-4361** — Sign In With Ethereum standard (Base implements this as SignInWithBase).
- **FOR UPDATE SKIP LOCKED** — Postgres row-locking pattern used in matchmaker + operator-worker claim paths.
- **H-01** — the challenge-window trust model upgrade to BattleArena (March 2026 audit).
- **MATCHMAKER_ROLE** — contract role granting `createBattle` permission.
- **operator_jobs** — durable outbox table for operator-worker tasks.
- **Power score** — sum of evolution-tier weights in a team (Evolved=1/Elite=2/Apex=3, range 3–9 for Evolved-or-better teams).
- **Purity score** — count of body parts where the dominant allele's class affinity matches the lobster's overall class (0–6).
- **RESOLVER_ROLE** — contract role granting `advanceRound` + `settle` permission.
- **V3 S1** — third major design iteration; Season 1 of the live game. Mining + Battle parallel economies; Power Matchmaking; admin dispute resolution.

- **X1, X2, X3, …** — finding IDs from the cross-cutting Codex sweeps. Live in `launch-blockers.md`.
- **A1, A2, A3, …** — finding IDs from the multi-PR campaign audit. Live in `launch-blockers.md`.

9.2 Useful one-liners

```
# Bun PATH
export PATH="$HOME/.bun/bin:$PATH"

# Typecheck a package
bun run --filter '@clawbada/engine' typecheck

# Engine tests
cd apps/engine && bun test

# Regenerate ABIs (after forge build)
cd packages/chain && bun run extract-abis

# Generate a migration
cd packages/db && bun run generate

# Foundry test
forge test --no-match-contract '...'   # filter expensive

# Quick git inspection
git log --oneline -20
git diff --stat HEAD~5..HEAD
git status
```

9.3 Where things live (file map cheat sheet)

CONCERN	FILE(S)
Battle math	`packages/game-logic/src/battle-sim.ts`
Class table	`packages/game-logic/src/classes.ts`, `packages/game-logic/src/constants.ts`
DNA codec	`packages/game-logic/src/dna.ts`
Gene inheritance	`packages/game-logic/src/gene-inheritance.ts`
ELO	`packages/game-logic/src/elo.ts`
Operator-worker	`apps/engine/src/operator/{worker,errors,types,jobs/*}.ts`
Indexer	`apps/indexer/src/watchers/battle-watcher.ts`, `apps/indexer/src/lib/event-processor.ts`
Matchmaker	`apps/api/src/lib/matchmaker/{match,tick,bucket}.ts`
API combat routes	`apps/api/src/routes/game/combat/{queue,battle-reads,battle-writes,index}.ts`
Frontend queue state	`apps/web/src/hooks/use-queue-state.ts`
Frontend battle UI	`apps/web/src/components/game/battle-moves.tsx`, `apps/web/src/app/(game)/battle/[id]/page.tsx`, `apps/web/src/app/(game)/game/battle/page.tsx`
Schemas	`packages/db/src/schema/*.ts` (battles, operator-jobs, agents, lobsters, teams, expeditions, marketplace, breeding, seasons, indexer, events)
Migrations	`packages/db/drizzle/00*_*.sql`

CONCERN	FILE(S)
Contracts	`contracts/*.sol`
Deploy scripts	`contracts/script/{Deploy,Configure,DeployHelpers}.s.sol`
Chain clients	`packages/chain/src/client.ts` (getPublicClient / getMatchmakerClient / getResolverClient / getOperatorClient)
ABIs	`packages/chain/src/abis/*.ts` (auto-extracted)

9.4 Open questions Hermes should help answer

These are real product/architecture decisions that have not been resolved:

1. **When does S1 launch?** Tied to contract deploy + frontend redesign + lobster art templates. No firm date.
2. **How is OpenClaw skill package distribution coordinated?** Needs contact with the OpenClaw / Bankr team.
3. **Mainnet operator wallet topology**: single matchmaker + single resolver + single admin? Or backup signers per role? Tracker #22 leans toward needing failover.
4. **Faucet activation date** and pre-mint timing. Need to align with #2/#3 deploy.
5. **Battle Replay (BattleResolver.replay() on-chain)** — S2 roadmap. When does Hermes prioritize this vs. growth features?

Part 10 — Closing note

Clawbada is a meaningful project. The architecture is sound. The hardest server-side work — closing the two P0 production blockers (X1 + X2) — is done. The remaining path is largely about execution discipline: deploy contracts, validate end-to-end, harden the deferred items, ship to mainnet.

Hermes inherits a working codebase with strong invariants, well-tracked technical debt, and a campaign discipline that should continue. The most valuable thing Hermes can do is **maintain that discipline while shipping faster than I could alone**. Use the operator-worker pattern wherever the same shape recurs ("off-chain decision needs operator-signed on-chain tx"). Run Codex sweeps after every meaningful change. Keep `launch-blockers.md` honest. Don't trust memory without verification.

Good luck.

— Outgoing engineer-of-record, 2026-05-19